

University Exam Scheduling as a CSP

PEAS, Agent Type, CSP Formulation, Backtracking and Forward Checking

Problem: Schedule 5 exams – A (Artificial Intelligence), B (Database Systems), C (Computer Networks), D (Operating Systems), E (Software Engineering) – into 3 slots (Slot1, Slot2, Slot3) so that the conflicting pairs A–B, A–C, B–C, B–D, C–D, C–E, D–E never share a slot. Slots are abbreviated S1, S2, S3 in tables.

a) PEAS Description (10 marks)

PEAS element	Description
Performance measure	• Zero conflicts: no two conflicting courses share a slot (hard constraint) • All 5 exams scheduled, each exactly once, using only Slot1–Slot3 • Fewer search steps / backtracks (computational efficiency) • (Optional soft goals) balanced slot usage, fewer students with back-to-back exams
Environment	University exam timetabling: 5 courses (A–E), 3 slots (one per day), a conflict list of 7 pairs from shared student enrolments. Fully observable, deterministic, static, discrete, single-agent, sequential (episodic assignments build one timetable).
Actuators	Assign (or un-assign) a slot to a course; output/publish the timetable; report failure if no valid schedule exists.
Sensors	Course list, slot list, enrolment/conflict data (the 7 conflicting pairs), and the current partial timetable (which courses already hold which slots).

b) Type of Agent (10 marks)

Best suited: Goal-based agent.

Agent type	How it works	Suitability for exam scheduling
Simple reflex	Acts on the current percept only (condition–action rules)	Unsuitable: a slot choice depends on earlier choices, so it needs memory of the partial timetable and look-ahead.
Model-based reflex	Keeps internal state, but still acts through fixed rules	Insufficient: tracks the timetable but has no explicit goal and cannot search/backtrack.
Goal-based	Searches/plans for action sequences that reach an explicit goal state	BEST FIT: the goal (complete, conflict-free timetable) is clear and binary.
Utility-based	Maximises a utility function over goal states	Not required: all valid timetables are equally acceptable here; useful only if soft preferences were added.

Justification:

- The task has an explicit goal state: every course holds a slot and no conflicting pair shares one. The agent must find a sequence of assignments that reaches it.

- Decisions are interdependent. Choosing Slot1 for A removes Slot1 from B and C, so the agent needs a model of the partial timetable and must look ahead and backtrack when a branch fails – a reflex agent cannot do this.
- Success is binary (valid / not valid). All conflict-free timetables are equally good, so no utility function is needed. A utility-based agent would only be justified if soft preferences (e.g. balanced days, gaps between exams) were added.
- A goal-based agent whose goal test and search are structured as a CSP solver (backtracking + inference) matches this problem directly.

c) CSP Formulation and Constraint Graph (15 marks)

Variables, Domains

Variable	Course	Domain
A	Artificial Intelligence	{Slot1, Slot2, Slot3}
B	Database Systems	{Slot1, Slot2, Slot3}
C	Computer Networks	{Slot1, Slot2, Slot3}
D	Operating Systems	{Slot1, Slot2, Slot3}
E	Software Engineering	{Slot1, Slot2, Slot3}

Constraints (binary, inequality)

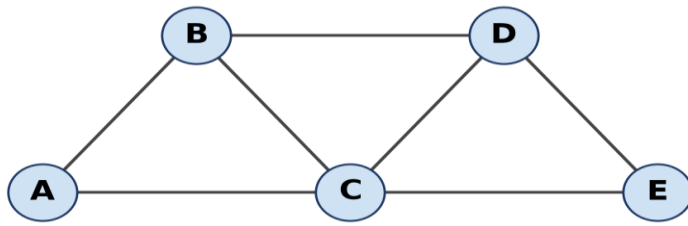
Conflicting pair	Constraint
A–B	$A \neq B$
A–C	$A \neq C$
B–C	$B \neq C$
B–D	$B \neq D$
C–D	$C \neq D$
C–E	$C \neq E$
D–E	$D \neq E$

In compact form: for every conflicting pair (X, Y), $\text{slot}(X) \neq \text{slot}(Y)$. Courses with no listed conflict (e.g. A–D, A–E, B–E) may share a slot.

Constraint graph

Vertices: {A, B, C, D, E}

Edges: {(A,B), (A,C), (B,C), (B,D), (C,D), (C,E), (D,E)} – 7 edges



Vertex	Neighbours	Degree
A	B, C	2
B	A, C, D	3
C	A, B, D, E	4
D	B, C, E	3
E	C, D	2

The graph consists of three triangles (A-B-C, B-C-D, C-D-E), so at least 3 colours (slots) are needed, and 3 suffice. C has the highest degree (4).

d) Plain Backtracking Search (25 marks)

Variable order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$. Value order: Slot1 $\rightarrow$ Slot2 $\rightarrow$ Slot3. A value is checked against already-assigned neighbours only (no look-ahead). Each value tried counts as one assignment attempt.

Step	Attempt	Result	Reason	Partial assignment
1	A=Slot1	OK	Consistent; assigned	A=S1
2	B=Slot1	Fail	Conflicts with A=Slot1	A=S1
3	B=Slot2	OK	Consistent; assigned	A=S1, B=S2
4	C=Slot1	Fail	Conflicts with A=Slot1	A=S1, B=S2
5	C=Slot2	Fail	Conflicts with B=Slot2	A=S1, B=S2
6	C=Slot3	OK	Consistent; assigned	A=S1, B=S2, C=S3
7	D=Slot1	OK	Consistent; assigned	A=S1, B=S2, C=S3, D=S1
8	E=Slot1	Fail	Conflicts with D=Slot1	A=S1, B=S2, C=S3, D=S1
9	E=Slot2	OK	Consistent; assigned	A=S1, B=S2, C=S3, D=S1, E=S2

Summary: 9 assignment attempts (5 successful, 4 failed), 0 backtracks. No variable ran out of values, so no consistent assignment had to be undone.

Final valid timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2.

Check: $A \neq B$ ($1 \neq 2$), $A \neq C$ ($1 \neq 3$), $B \neq C$ ($2 \neq 3$), $B \neq D$ ($2 \neq 1$), $C \neq D$ ($3 \neq 1$), $C \neq E$ ($3 \neq 2$), $D \neq E$ ($1 \neq 2$) – all satisfied. Exam days: Slot1 = AI and Operating Systems; Slot2 = Database Systems and Software Engineering; Slot3 = Computer Networks.

e) Backtracking with Forward Checking (20 marks)

Same variable and value order. After each assignment $X = v$, v is removed from the domain of every unassigned neighbour of X . If any domain becomes empty, the assignment is rejected immediately (wipe-out).

Step	Assign	Result	Values pruned	Domains of unassigned variables after step
0	(start)	—	—	A: {S1,S2,S3} B: {S1,S2,S3} C: {S1,S2,S3} D: {S1,S2,S3} E: {S1,S2,S3}
1	A=Slot1	OK	Pruned B-Slot1, C-Slot1	B: {S2,S3} C: {S2,S3} D: {S1,S2,S3} E: {S1,S2,S3}
2	B=Slot2	OK	Pruned C-Slot2, D-Slot2	C: {S3} D: {S1,S3} E: {S1,S2,S3}
3	C=Slot3	OK	Pruned D-Slot3, E-Slot3	D: {S1} E: {S1,S2}
4	D=Slot1	OK	Pruned E-Slot1	E: {S2}
5	E=Slot2	OK	No pruning	(no unassigned variables left)

Walk-through:

- **A = S1:** removes S1 from B and C $\rightarrow$ B {S2,S3}, C {S2,S3}.
- **B = S2:** removes S2 from C and D $\rightarrow$ C {S3}, D {S1,S3}. (A is already assigned.)
- **C = S3:** removes S3 from D and E $\rightarrow$ D {S1}, E {S1,S2}.
- **D = S1:** removes S1 from E $\rightarrow$ E {S2}.
- **E = S2:** the only value left is consistent; all variables assigned.

Summary: 5 assignment attempts, 0 failures, 0 backtracks, 7 value prunings. The domains of C, D and E became singletons, so no wrong value was ever tried.

Final valid timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2 (identical to part d).

f) Comparison (20 marks)

Measure	Plain backtracking	Backtracking + forward checking
Assignment attempts (values tried)	9	5
– successful assignments	5	5
– failed attempts (conflict found)	4 (B=S1, C=S1, C=S2, E=S1)	0 (those values were already pruned)

Measure	Plain backtracking	Backtracking + forward checking
Backtracks (undo of a consistent assignment)	0	0
Domain wipe-outs detected	—	0
Values pruned by look-ahead	—	7 (B1, C1, C2, D2, D3, E3, E1)
Final timetable	A=S1, B=S2, C=S3, D=S1, E=S2	A=S1, B=S2, C=S3, D=S1, E=S2

Efficiency

- Attempts: forward checking needs 5 attempts versus 9 – about 44% fewer. It makes exactly one attempt per variable, which is the minimum possible for 5 variables.
- Backtracks: both runs have 0 backtracks here. The value and variable order happen to lead straight to a solution, so the advantage of forward checking shows up only in failed attempts, not in backtracks.
- Overhead: forward checking does extra work after every assignment (updating neighbour domains: 7 removals in total). This is cheap compared with the 4 wasted attempts it avoids, and the saving grows on larger problems.

Why forward checking reduces search effort

- It propagates each assignment to the unassigned neighbours immediately, so conflicting values are removed before they are ever tried. Plain backtracking discovers each conflict only when it reaches that value (B=S1, C=S1, C=S2, E=S1).
- It detects dead ends early. If a neighbour's domain becomes empty, the current assignment is rejected at once, instead of exploring a doomed subtree until it fails deeper down. That does not occur in this instance but is the main source of savings in harder ones.
- It exposes forced moves: C, D and E each reached a single remaining value, so the rest of the search was deterministic.
- In the worst case both methods remain exponential (up to $3^5 = 243$ complete assignments), but forward checking prunes whole branches and is never worse in attempts than plain backtracking with the same orderings. Adding MRV/degree ordering or arc consistency would improve it further.

Conclusion: both methods give the same valid timetable (A=Slot1, B=Slot2, C=Slot3, D=Slot1, E=Slot2). Forward checking reaches it with fewer attempts and no wasted conflict checks, so it is more efficient. It is also unique up to renaming of slots: $3! = 6$ valid timetables exist, because A, B, C must all differ, which forces D = A's slot and E = B's slot.